OUTAGE DISSECTIONS • DATABASE PERFORMANCE • QUERY OPTIMIZER BUGS

How a Single MySQL Index Broke GitHub: Query Optimizer Inversion

Curated from Arpit Bhayani's Architecture Dissection Series • Core Focus: MySQL Cost-Based Optimizer & Index Flips

EXECUTIVE ARCHITECTURAL ABSTRACT

Indexes are generally assumed to be safe, additive performance improvements. In this notable GitHub production incident, adding an innocent secondary index on a multi-million-row table caused the **MySQL Cost-Based Optimizer (CBO)** to flip its execution plan. Instead of executing a fast, 1ms B+ tree seek using the existing composite index, MySQL opted to scan millions of records in reverse chronological order to satisfy an `ORDER BY ... LIMIT` clause. The 10,000x latency explosion drove server CPU to 100%, exhausting connection pools and causing a platform-wide blackout.

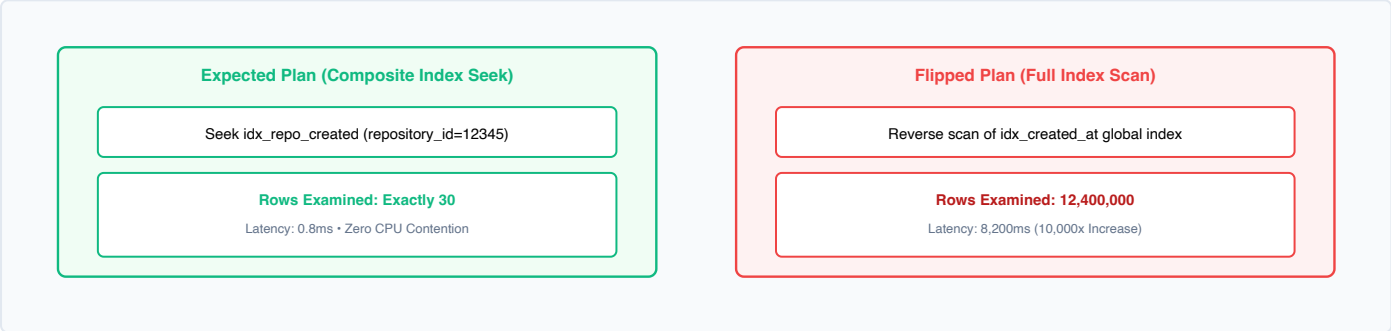
1. THE ANATOMY OF THE QUERY PLAN INVERSION

Consider a high-frequency dashboard query executed thousands of times per second:

```
SELECT id, title, state, created_at FROM issues
WHERE repository_id = 12345 ORDER BY created_at DESC LIMIT 30;
```

- **Pre-Migration State:** Existing composite index `(repository_id, created_at)` performed a direct seek on the repository ID and extracted the first 30 pre-sorted records. Latency: **0.8ms**.
- **The Schema Migration:** An engineer added a secondary index on `(created_at)` alone to support a global event timeline.
- **The Optimizer Inversion:** The optimizer evaluated the cost of sorting vs. scanning. Because of `LIMIT 30`, it incorrectly calculated that scanning `idx_created_at` in reverse order would quickly yield 30 matching rows, avoiding a filesort.

2. VISUALIZING EXPECTED SEEK VS. FLIPPED SCAN



3. THE MECHANICS OF THE COST-BASED OPTIMIZER TRAP

Why MySQL chose the disastrous execution path over the optimal one:

1. The LIMIT Cost Heuristic Discount

MySQL's optimizer assumes that data matching any WHERE clause is uniformly distributed across the table. When evaluating `LIMIT 30`, it estimates that scanning the pre-sorted index will encounter 30 matching rows within just a few hundred page reads, giving it an artificially low cost estimate.

2. Skewed Data Distribution Reality

In reality, repository 12345 was a low-activity repository. Its 30 most recent rows were buried deep across 12 million index leaf nodes. MySQL scanned millions of rows to find 30 matches, thrashing RAM and driving CPU utilization to 100%.

4. CASCADING CONNECTION & HEALTH CHECK COLLAPSES

The 10,000x query latency explosion cascaded through the infrastructure tiers:

- **Connection Exhaustion:** 5,000 application worker threads hung waiting for queries to complete, instantly hitting MySQL's `max_connections` ceiling.
- **ProxySQL Queue Breaches:** Ingress proxies buffered in-flight queries until internal buffer pools were exhausted, returning HTTP 500 errors.
- **False Heartbeat Failover:** Monitoring probes could not establish connections to execute heartbeats, triggering false Orchestrator master promotions that worsened downtime.

5. TECHNICAL MITIGATIONS: DEFEATING OPTIMIZER INVERSIONS

To prevent optimizer flips, production architectures enforce strict query constraints:

1. Explicit Index Hints (FORCE INDEX)

```
-- Force optimal index selection explicitly in ORM queries
SELECT id, title, state, created_at
FROM issues FORCE INDEX (idx_repo_created)
WHERE repository_id = 12345 ORDER BY created_at DESC LIMIT 30;
```

2. MySQL 8.0 Invisible Indexes

Create new indexes as `INVISIBLE`. The engine maintains index B+ trees without the optimizer considering them. Validate execution plans using session flags before enabling globally:

```
CREATE INDEX idx_created_at ON issues (created_at) INVISIBLE;
SET SESSION optimizer_switch = 'use_invisible_indexes=on'; -- Test safely
ALTER TABLE issues ALTER INDEX idx_created_at VISIBLE;      -- Enable once verified
```

6. EXECUTION PLAN COMPARISON: COMPOSITE VS. FLIPPED SCAN

Metric	Composite Index Seek (idx_repo_created)	Flipped Index Scan (idx_created_at)
Type in EXPLAIN	`ref` (Indexed B+ Tree Range)	`index` (Full Index Scan)
Rows Examined	**30 rows**	**12,400,000 rows**
Execution Time	**0.8 milliseconds**	**8,200 milliseconds**
Buffer Pool Thrashing	Zero (Pages already in RAM)	High (Evicts hot pages to read cold index)
CPU Impact	< 0.1% CPU core load	100% saturation across all cores

7. PRODUCTION ENGINEERING TAKEAWAYS

1. **Indexes Can Break Production:** Never treat index creation as purely additive. Adding an index changes the cost calculation of every query targeting that table.
2. **Beware of ORDER BY ... LIMIT Optimization Traps:** The optimizer's attempt to avoid sorting by using an index matching the ORDER BY clause is a known hazard when combined with non-uniformly distributed WHERE filters.
3. **Automate EXPLAIN Diffing in CI/CD:** Run query plan regressions before releasing schema migrations. If any query changes from `ref` or `range` to `index` or `ALL`, halt deployment immediately.